

ASSIGNMENT 2

— Build Bug Report Management Application with UI —

1. Overview

In this assignment, you will extend and improve the project from Assignment 1, building a web-based Bug Report management application using either EJS or Handlebars (pick one).

In Assignment 2, a User entity will be added to assign users to fix bug reports

After completing the assignment, you will have a web-based Bug Report management application with an intuitive interface.

2. Main requirements:

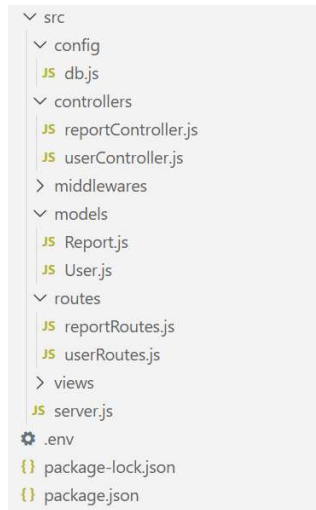
- Integrate APIs from Assignment 1 to process data from MongoDB.
- Display the list of Reports and Users on the web interface
- Add, edit, and delete Reports and Users through the user interface.
- Use template engine (EJS, Handlebars) to render interface.
- Use Bootstrap or CSS to create beautiful and user-friendly interface.

3. Implementation Steps:

Step	Task	Description
Step 1	Initialize Node.js project named: assignment2_<memberCode> (lowercase). (ex: assignment2_sangnqce150621)	Create a new project using <code>npm init -y</code> and install dependencies.
Step 2	Prepare the environment	Install Express.js, EJS, Handlebars, Bootstrap.
Step 3	Directory structure	Create routes, views, public, controllers folders
Step 4	Configure Express and EJS or Handlebars	Set up template engine, middleware
Step 5	Build API & Routes	Write routes to manage Report Bug
Step 6	Create user interface (UI)	Use EJS/Handlebars to display list and input form
Step 7	Test the application	Run and test CRUD functions through the interface.

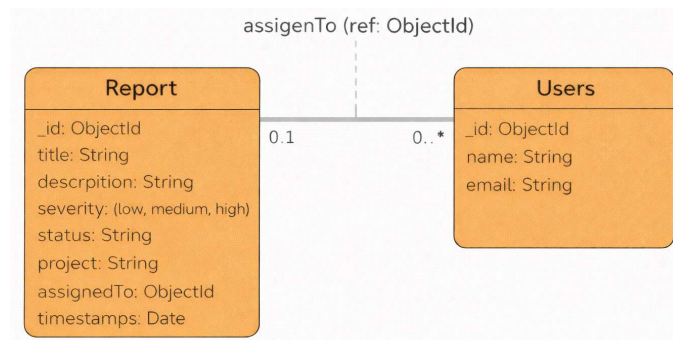
3. Assignment Requirements:

a) Project Structure (MVC Model): Organize your source code following the MVC structure for clarity and scalability.



- **Model:** Define data schema and validation (using Mongoose).
- **Controller:** Handle business logic and database operations.
- **Router:** Manage endpoints and link them to controllers.

b) Create a *Report* and *User* model with the following fields:



Report.js

Field	Type	Description
title	String (required)	Title of the bug report.
description	String (required)	Detailed description of the issue.
severity	String (enum); default: "low"	One of: low, medium, high
status	String (enum); default: "open"	One of: open, in-progress, resolved, closed.
project	String	Project name or code.
timestamps	Date	Auto-generated <code>createdAt</code> and <code>updatedAt</code> .

Field	Type	Description
assignedTo	type: mongoose.Schema.Types.ObjectId, ref: ' User ', required: true	Link data with User ID

User.js

Field	Type	Description
name	String (required)	Name of the User.
email	String (required, unique)	Email is not allowed to be duplicated
createdAt	Date default: Date.now	createdAt defaults to the current date

c) MongoDB Connection

- Use **local MongoDB**:
mongodb://127.0.0.1:27017/bug_reports
- Store the connection string in a **.env** file as:
MONGODB_URI=<your_connection_string>
PORT=8000

d) Create REST API Specification:

Reports

HTTP Method	Endpoint	Function
GET	/api/reports	Get all reports (optionally filter by status, severity, or project).
GET	/api/reports/:id	Get a specific report by ID.
POST	/api/reports	Create a new bug report.
PUT/PATCH	/api/reports/:id	Update an existing bug report.
DELETE	/api/reports/:id	Delete a bug report.

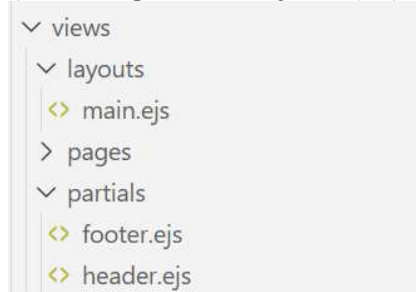
Users

HTTP Method	Endpoint	Function
GET	/api/users	Get all users
GET	/api/users/:id	Get a specific users by ID.
POST	/api/users	Create a new users .
PUT/PATCH	/api/users/:id	Update an existing users.
DELETE	/api/users/:id	Delete a users.

Requirements:

- Return proper HTTP status codes (200, 201, 404, 500, etc.).

e) Creating User Interfaces (UI) with EJS/Handlebars using the following structure:



Use Bootstrap to create a better interface

- Create layout for application with navbar with User, Report
- Creating CRUD Interfaces for Bug Reports
- Creating CRUD Interfaces for Users

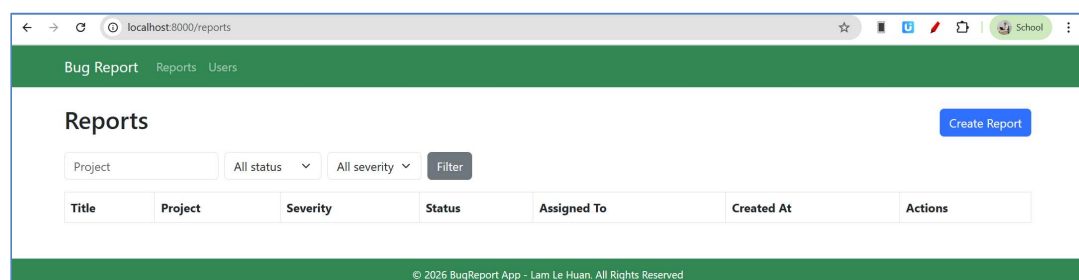
Note: Require populate **assignedTo** when rendering report list (to show user name)

The application interface is as follows:

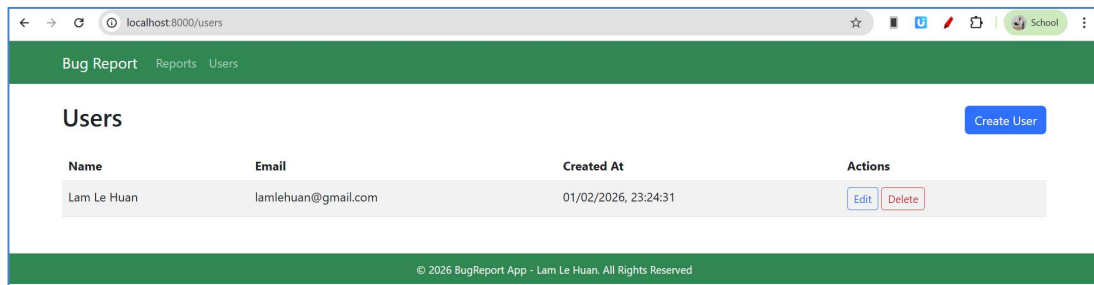
- Using **Partials** in EJS/ Handlebars to create **header** and **footer** (write the student's name correctly)



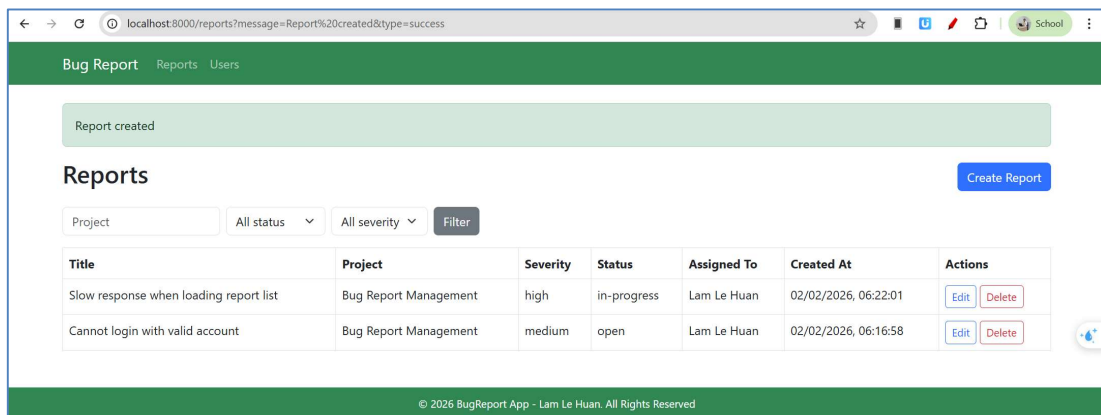
- Accessing the homepage <http://localhost:8000/> will redirect you to the list of reports.



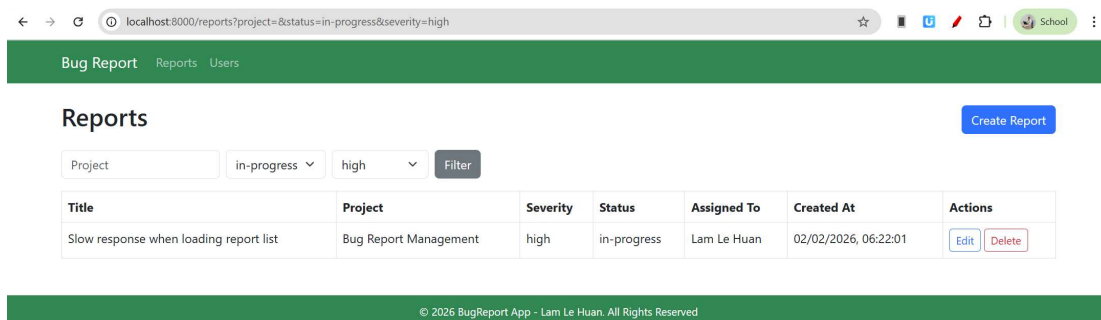
- Select the **Users** menu to add a new user (using the student's own information for the assignment)



- Create a bug report with optional information.



- Use filters to filter bug report information.



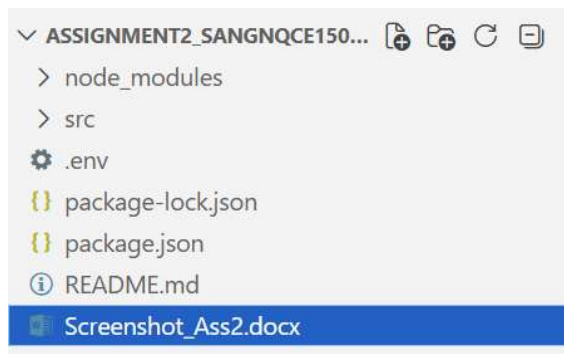
f) Test the Application

- Run the server: **npm run dev**
- Open the browser and check <http://localhost:8000/>
- Test the API and UI to ensure they work as described.

4. Submission Requirements:

a) Source Code Folder: Include full project ([remove node_modules](#)) with comments at the top of each file.

b) Documentation File (.docx): Name the file [Screenshot_Ass2.docx](#). Download the file, open it, and paste the screenshot as instructed into the correct location. Total of 10 screenshots. After you're done, copy it to the root project folder before submitting your assignment.



c) **Submission Format:** Submit as .zip/.rar named: assignment2_<memberCode>.zip

5. Evaluation Criteria (Total 10 Points)

- Organizing the project according to the structure and naming conventions earns 2 points.
- Each screenshot earns 1 point. Total 8 points.

→ **Total 10 points.**

6. Important Notes:

- ✗ Incomplete submissions or missing documents will lose points.
- ✗ Copying from another student or gene code from AI will get 1 point.
- ☑ Double-check your project runs correctly before submitting.